

PA1 Report

R15943080 汪煒杰

Problem2(a)

The two-bit unsigned multiplier was implemented in `mul.blif` and submitted separately.

Problem2(b)

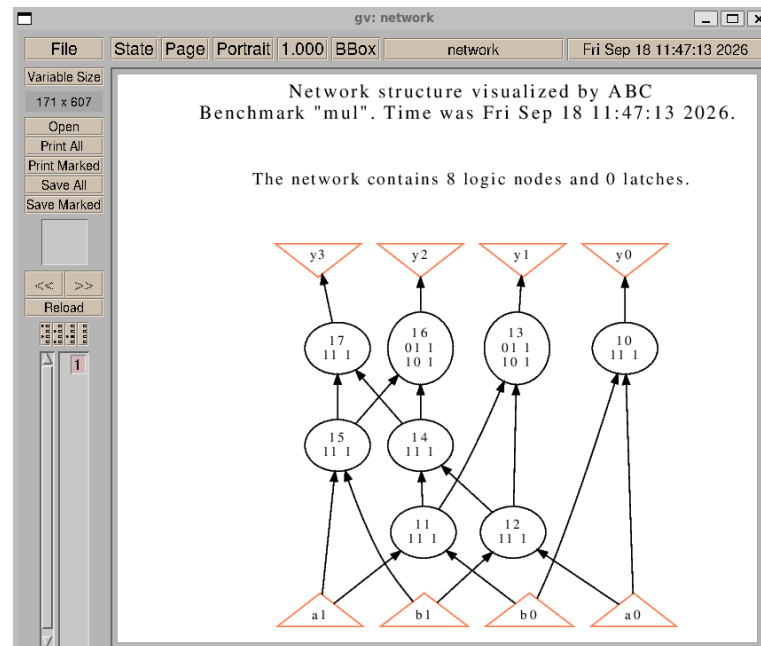
2-1, 2-2

```
abc 01> read mul.blif
abc 02> print_stats
mul          : i/o = 4/ 4 lat = 0 nd = 8 edge = 16 cube = 10 lev = 3
abc 02> 
```

2-3

```
abc 02> show
abc 02> Warning: Missing charsets in String to FontSet conversion

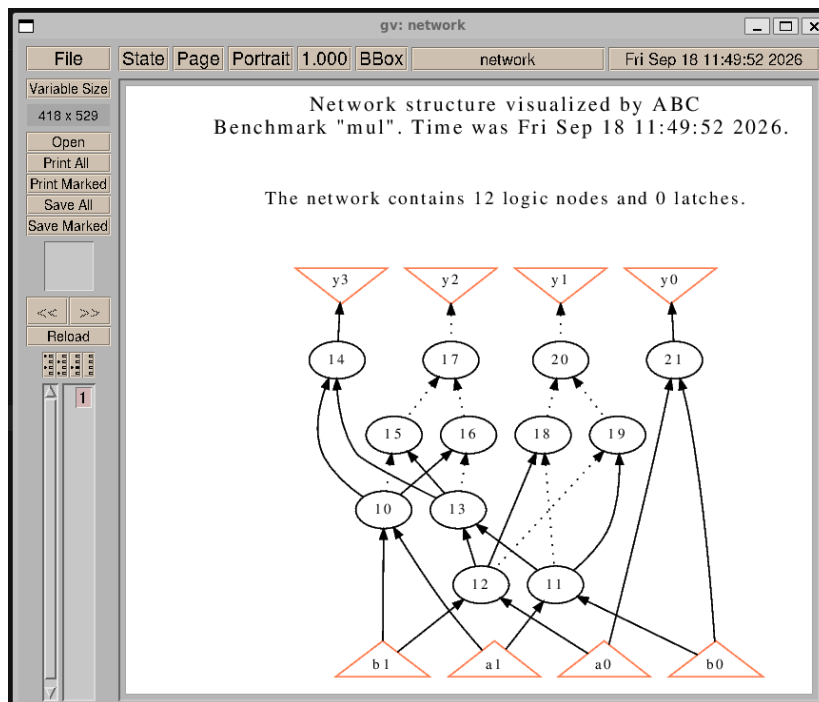
```



2-4,2-5

```
strash
abc 03> show
abc 03> Warning: Missing charsets in String to FontSet conversion

```

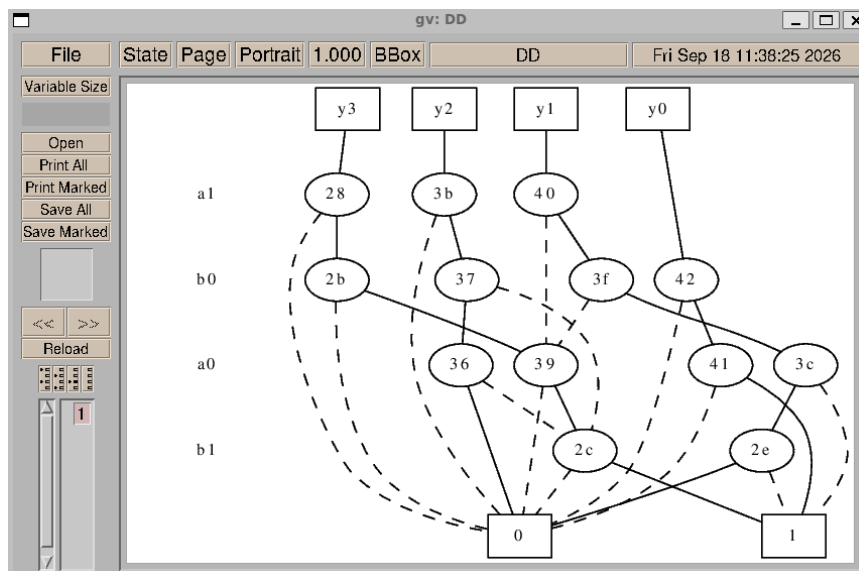


2-6, 2-7

```

abc 01> read mul.blif
abc 02> print_stats
mul          : i/o =  4/  4 lat =  0 nd =  8 edge =  16 cube =  10 lev = 3
abc 02> show
abc 02> Warning: Missing charsets in String to FontSet conversion
strash
abc 03> show
abc 03> Warning: Missing charsets in String to FontSet conversion
collapse
abc 04> show_bdd -g
abc 04> Warning: Missing charsets in String to FontSet conversion

```



Problem3

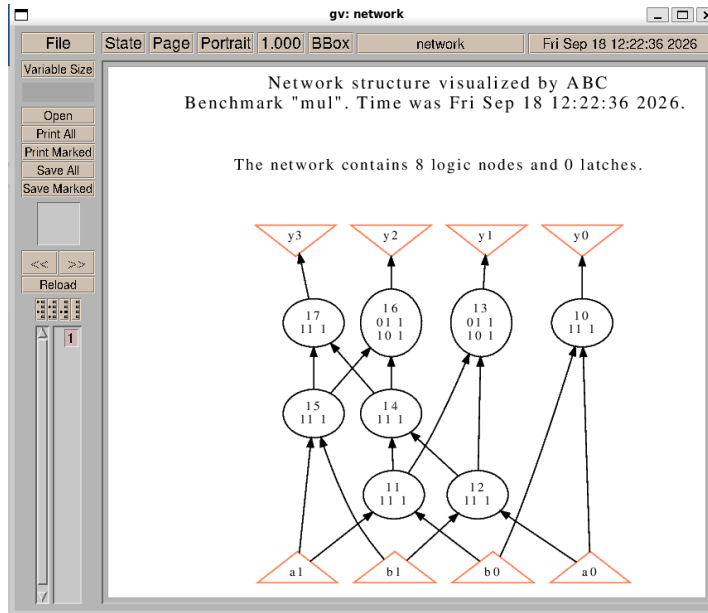
(a)-1

AIG

```

abc 01> read mul.blif
abc 02> aig
abc 02> print_stats
mul                               : i/o =  4/  4 lat =  0 nd =  8 edge =  16 aig =  12 lev = 3
abc 02> show
abc 02> Warning: Missing charsets in String to FontSet conversion
abc 02>

```

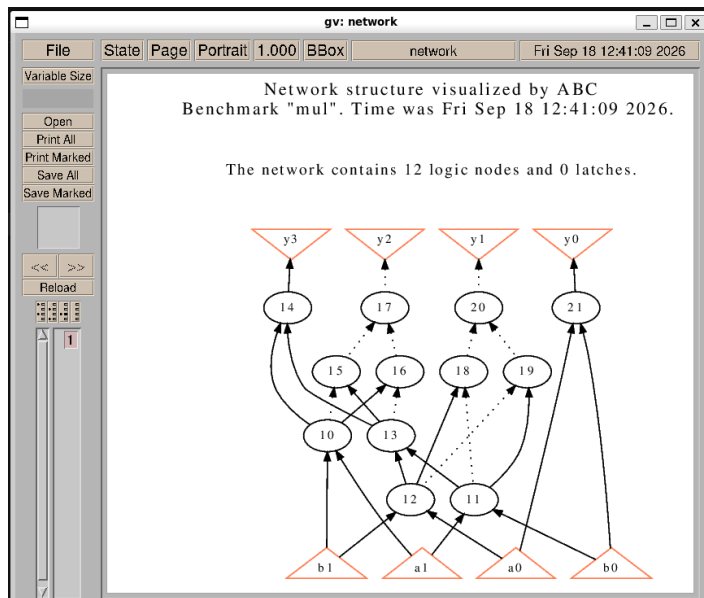


Strash

```

abc 01> read mul.blif
abc 02> strash
abc 03> print_stats
mul                               : i/o =  4/  4 lat =  0 and =  12 lev = 4
abc 03> show
abc 03> Warning: Missing charsets in String to FontSet conversion

```



AIG vs. Structurally Hashed AIG:

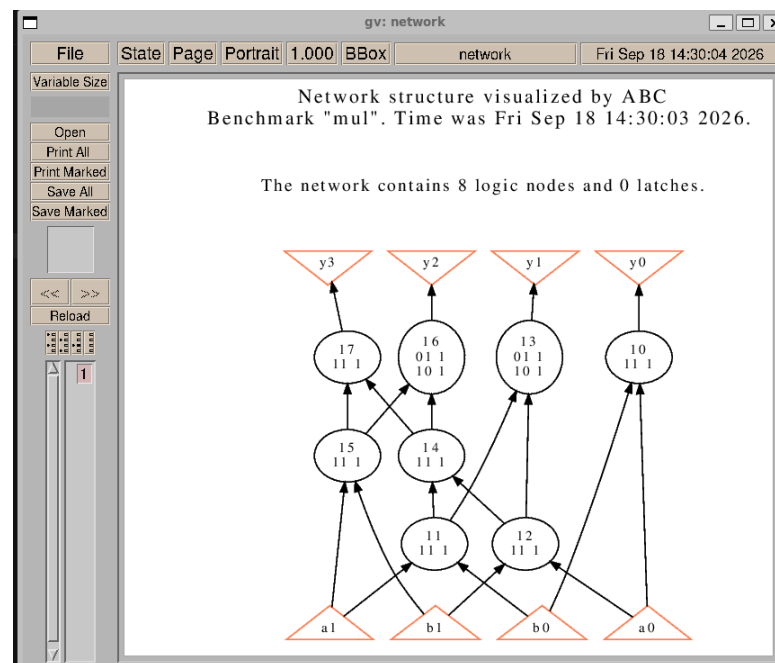
After the `aig` command, the circuit remains a multi-level logic network with 8 logic nodes, so its network visualization is unchanged. The Boolean function of each logic

node is represented internally as an AIG, and `print_stats` reports 12 AIG nodes in total. In contrast, `strash` converts the entire network into one structurally hashed AIG. The original logic-node boundaries are removed, and the resulting AIG contains 12 AND nodes. Therefore, the network structure and logic depth change after `strash`.

(a)-2

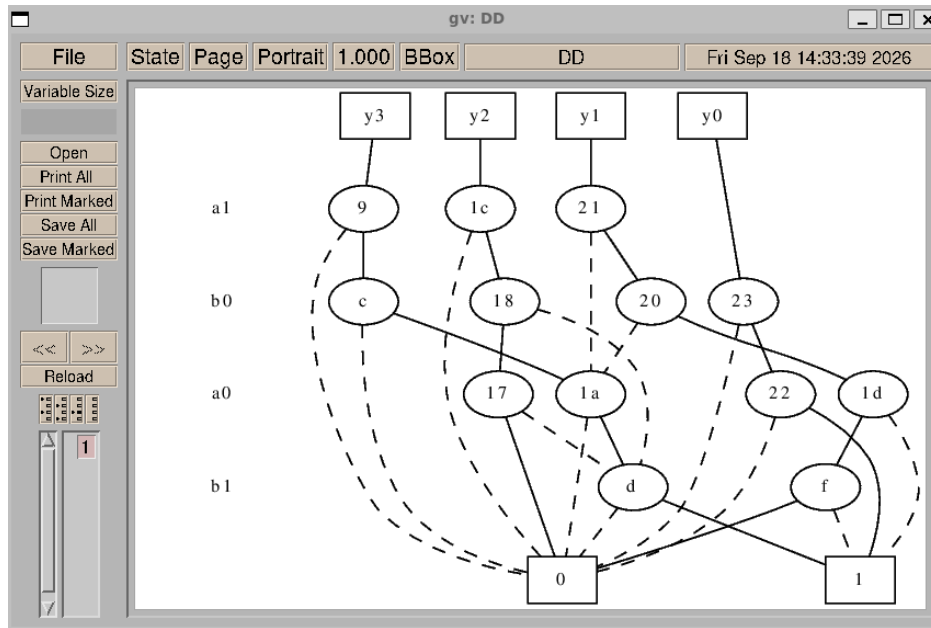
Logic Network in BDD

```
abc 01> read mul.blif
abc 02> bdd
abc 02> print_stats
mul          : i/o =  4/  4  lat =  0  nd =   8  edge =  16  bdd =  16  lev = 3
abc 02> show
abc 02> Warning: Missing charsets in String to FontSet conversion
```



Collapsed BDD

```
abc 01> read mul.blif
abc 02> collapse
abc 03> print_stats
mul          : i/o =  4/  4  lat =  0  nd =   4  edge =  14  bdd =  14  lev = 1
abc 03> show_bdd -g
abc 03> Warning: Missing charsets in String to FontSet conversion
```



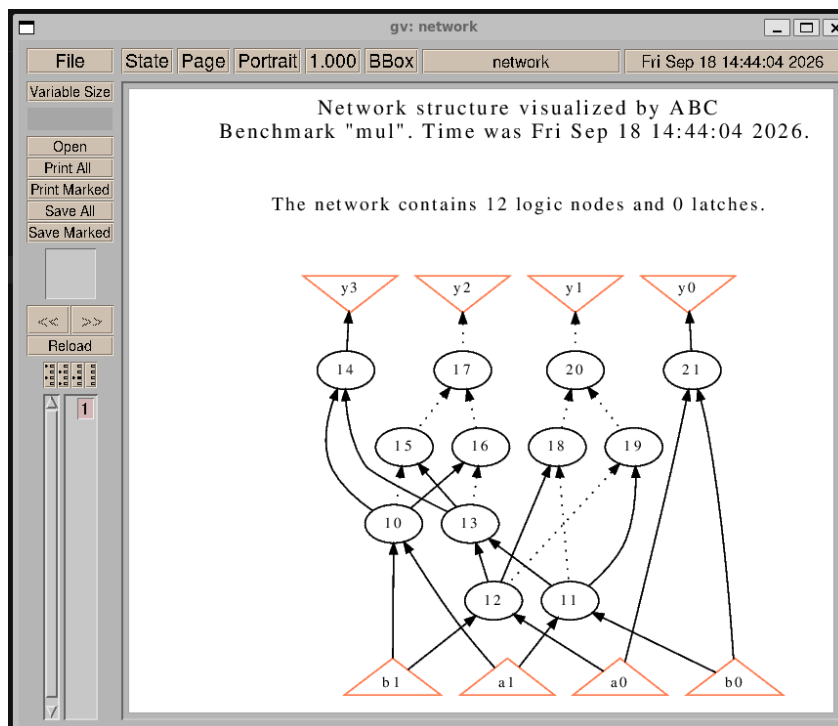
BDD vs. Collapsed BDD:

After the `bdd` command, the circuit remains a multi-level logic network with 8 logic nodes and 3 levels. Only the Boolean function of each logic node is represented internally using BDDs, so the network visualization remains unchanged. The `print_stats` result reports 16 BDD nodes in total. In contrast, `collapse` removes the intermediate logic-node boundaries and represents each primary output directly as a function of the primary inputs. As a result, the network is reduced to 4 logic nodes, one for each output, with a logic depth of 1. The collapsed representation contains 14 BDD nodes.

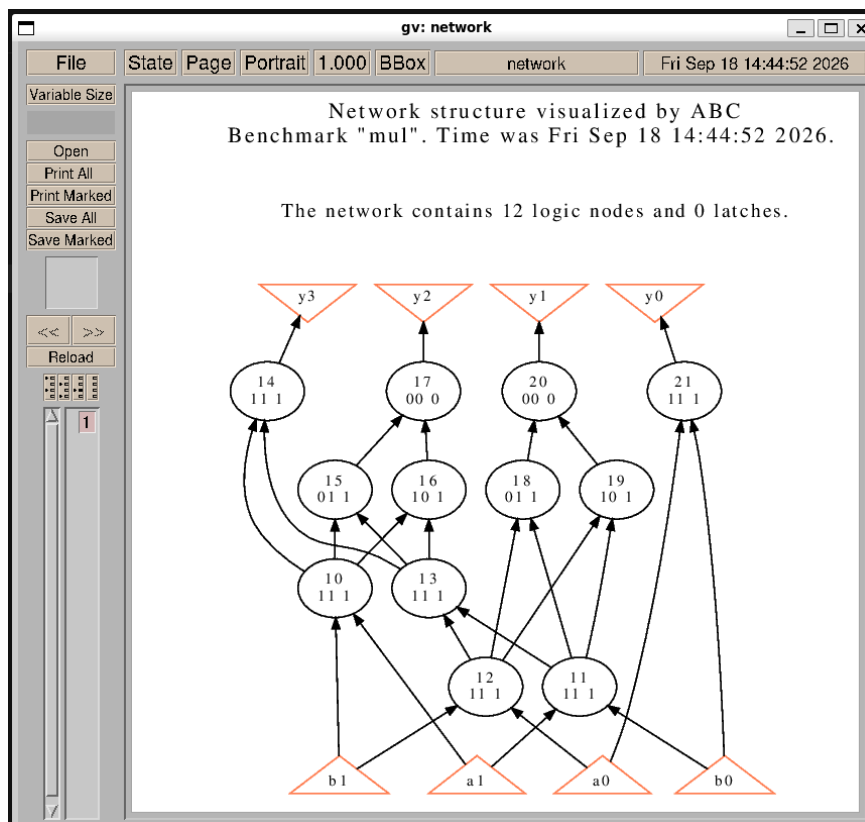
(b)

```
abc 01> read mul.blif
abc 02> strash
abc 03> print_stats
mul          : i/o = 4/ 4 lat = 0 and = 12 lev = 4
abc 03> show
abc 03> Warning: Missing charsets in String to FontSet conversion
logic
abc 04> print_stats
mul          : i/o = 4/ 4 lat = 0 nd = 12 edge = 24 cube = 12 lev = 4
abc 04> show
abc 04> Warning: Missing charsets in String to FontSet conversion
```

Structurally Hashed AIG after `strash`



Logic Network with SOP Node Functions (`logic`)



AIG to SOP Logic Network:

Starting from the structurally hashed AIG generated by `strash`, the `logic` command

converts the circuit into a logic network whose node functions are represented in SOP form. After the conversion, `print_stats` reports 12 logic nodes and 12 cubes, and the SOP cubes are visible inside the nodes in the network visualization.